



Waipapa
Taumata Rau
**University
of Auckland**

CS340 – Tutorial Week 5

Midterm Review: Paging, Page Replacement &
Past-Paper Practice

➤➤➤ August 19

➤➤➤ Connor Williams

Plan for today



Less new lecture content this week; most of the hour is past-paper practice for the midterm, plus a quick wrap-up of Assignment 1's last task.

Quick recap: paging & multi-level page tables

Quick recap: TLB & page replacement algorithms

Assignment 1 due soon!

Midterm practice: past-paper deep dives

Wrap-up & questions



Attendance

Scan the QR code here to
mark attendance

Paging · TLB · Page Replacement

Quick Recap: This Week's Lectures

Paging Splits an Address into VPN + Offset

Address space = one process's private view of memory (code, heap, stack)

The MMU translates every virtual address before RAM is touched

Paging divides memory into fixed-size blocks: a page (virtual) / a frame (physical)

Virtual address = VPN (selects a page-table entry) + offset (selects a byte)

The page table swaps VPN → PPN; the offset is copied across unchanged

A flat single-level table reserves a PTE for every possible VPN; even unused ones

With 4 KiB pages, the last three hexadecimal digits are the 12-bit offset.

Virtual address	VPN lookup	PPN	Physical address
0x0 AB0	0x0 → 0x1	0x1	0x1 AB0
0x1 FA0	0x1 → 0x4	0x4	0x4 FA0
0x2 884	0x2 → 0x3	0x3	0x3 884
0x3 2D0	0x3 → 0x7	0x7	0x7 2D0

The bold offset is copied unchanged into the physical address.

Multi-Level Tables Save Space; the TLB Saves Time

Multi-level page tables

A page-sized table holds 512 entries (9 index bits) with 4 KiB pages / 8 B PTEs

A level entry either points to the next table, or marks the region unmapped (no lower table is allocated for an unused region)

Quick facts for 30-bit VA / 512 B pages / 8 B PTEs:
single-level table = 2^{21} PTEs · each multi-level table = 512 B · minimum 4 levels

TLB (Translation Lookaside Buffer)

Small hardware cache of recent VPN → PPN translations

EAT = Effective Access Time

Hit: skip the walk — $EAT = t_{TLB} + M(\text{data})$

Miss: full walk — $EAT = t_{TLB} + L \cdot M(\text{levels}) + M(\text{data})$

Worked example (2-level, 80% hit, TLB=10ns, mem=100ns): Hit=110ns, Miss=310ns → EAT=150ns

Context switch: invalidate stale entries, or tag translations with an ASID

Page Replacement: Five Policies at a Glance

On a fault with no free frame, a policy picks the victim.

Optimal (needs the future): unbeatable lower bound.

FIFO can get WORSE with more frames: Belady's anomaly (3→9, 4→10 faults).

LRU approximates Optimal via real recency; exact LRU is too costly to track.

Clock approximates LRU cheaply with one reference bit per page.

Policy	Information used	Main role
Optimal	future references	lower-bound benchmark
Random	none	simple baseline
FIFO	arrival order	simple, but can behave poorly
LRU	exact recency	good idea, expensive to maintain
Clock	reference bit	practical approximation of recency

The best practical policy balances fault rate against the cost of tracking the information needed to make each decision.

Due Friday midnight

Wrapping Up Assignment 1



Assignment 1: Reflection Reminder

Reflection (2 pts)

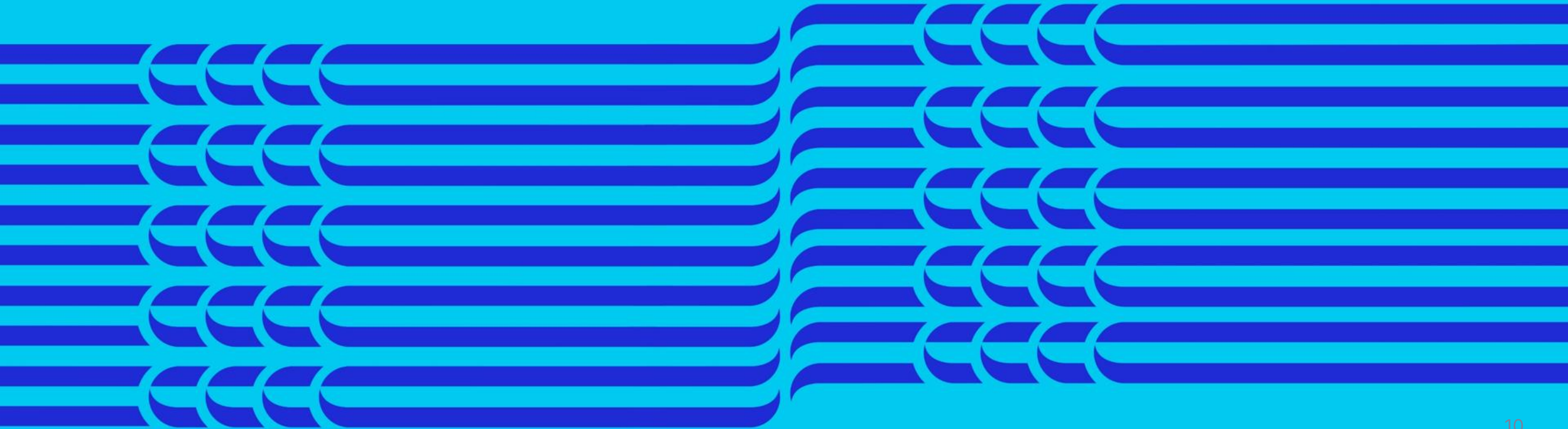
Your submission should include a short reflection discussing the assignment (at most **one page**).

In your reflection, you should discuss what you found **difficult** during the assignment, what you **learned**, and any **feedback** you would like to share.

Add your report as a single pdf file named **reflection.pdf** in the root folder.

Work through these with the person next to you

Midterm Practice: Past Papers



PQ1 - Which Output Can't Happen?

```
int main(void) {  
    if (fork() == 0) {  
        printf("B"); exit(0);  
    }  
    if (fork() == 0) {  
        printf("C"); exit(0);  
    }  
    wait(NULL);  
    printf("A");  
    return 0;  
}
```

Which of these outputs CANNOT occur?

- A. CAB
- B. ABC
- C. BCA
- D. More than one of these three is impossible
- E. All three are possible

PQ1 - Which Output Can't Happen?

Answer: B — "ABC" is impossible.

Only ONE `wait(NULL)` reaps whichever child exits FIRST, the other keeps running independently, so its letter can land before or after "A". But the reaped child must already have printed (print happens-before its exit, which happens-before `wait()` returning) — so "A" can never be first. CAB and BCA are both fine; ABC is not.

PQ2 - How Many Page-Table Bytes So Far?

30-bit VA, 512 B pages (9-bit offset)
8 B PTEs -> 64 entries/table (6-bit index)
-> minimum 4 levels

Page table bit widths, top -> bottom:
3, 6, 6, 6 bits

Note: every allocated table (even the smaller root) occupies one whole 512 B physical frame once allocated, the same way real hardware allocates page-table storage in whole pages.

Program has only ever touched:

0x08418812, 0x08418A1A,
0x0842021F

How many bytes of page table have been allocated so far?

- A. 4096
- B. 3584
- C. 4608
- D. 2560
- E. 3072

PQ2 — How Many Page-Table Bytes So Far?

Answer: D — 2560 bytes. (5 tables * 512 bytes per table = 2560 bytes)

All 3 addresses share one root table ⇒ 1 root table

All addresses share the same L2 table in the root PTE ⇒ 1 L2 table

All addresses share the same L1 table in the same L2 PTE ⇒ 1 L1 table

Two entries share an L0 table (000011), ⇒ 2 L0 tables

one uses a different table (000100) Total = 5 tables

0x08418812 = 0b 00001 000010 000011 000100 000010010

0x08418A1A = 0b 00001 000010 000011 000101 100011011

0x0842021F = 0b 00001 000010 000100 000001 000011111

(same root) (same L2) (same L1)(two different L2)

PQ2 continued - Min & Max Pages for 10 Data Pages

Same setup as before: 30-bit VA, 512 B pages, 8 B PTEs, 4 levels, root has only 8 entries.

Your program maps exactly 10 data pages (code + data).

What are the minimum and maximum number of pages allocated in total (data pages + page-table pages)?

- A. 13, 41 B. 13, 31 C. 14, 41 D. 14, 39 E. 13, 21

PQ2 continued — Min & Max Pages for 10 Data Pages

Answer: D — minimum 14, maximum 39

Minimum: all 10 data pages can share ONE L0 table (it holds up to 64 entries) $\rightarrow 1 \text{ root} + 1 \text{ L2} + 1 \text{ L1} + 1 \text{ L0} = 4 \text{ table pages} + 10 \text{ data pages} = 14$

Maximum: the root has only 8 entries, capping distinct L2 branches at 8 — but L1 and L0 tables aren't capped (up to 10 each, one per data page) $\rightarrow 1 \text{ root} + 8 \text{ L2} + 10 \text{ L1} + 10 \text{ L0} = 29 \text{ table pages} + 10 \text{ data pages} = 39$

6 of the 8 L2 tables only have one L1 page present in their PTE's, while two of them have two entries each to fit the full 10 L1 tables across the 8 L2 tables.

PQ3 — Which Scheduler Design for These Workers?

A multi-core scientific simulation launches a fixed number of long-running worker processes at startup. Each worker repeatedly reuses much of its own recent data, and the workers stay similarly busy for most of the run.

Design A: one global ready queue shared by all CPUs

Design B: one ready queue per CPU, with no movement between CPUs

Design C: one ready queue per CPU, with frequent movement of work between CPUs

Which design is most appropriate?

PQ3 — Which Scheduler Design for These Workers?

Answer: B — one ready queue per CPU, no movement

Each worker keeps reusing its own recently-computed data → it wants to stay on the SAME CPU to keep that data warm in cache (processor/cache affinity)

Designs A (global queue) and C (frequent migration) both risk moving a worker to a different CPU on every reschedule — cold cache, for no gain

Migration/load-balancing (C) earns its cost only when load is UNEVEN — here the workers are explicitly "similarly busy", so it isn't



Waipapa
Taumata Rau
**University
of Auckland**

Thank you!